Year: 2026-2027 **Sem V-NEP**

Course: Agile Software Development & DevOps Lab

Date of Performance: 04/09/2026 **Date of Submission: 12/09/2026**

Class: T1

Student Roll No: 24010014 **Student Name: Viraj Shyam Bhiwandkar**

EXPERIMENT – 7

AIM : To implement monitoring using Prometheus / Grafana / Nagios.

LO MAPPED : LO5.

LO5 : Implement ML lifecycle management and automated workflows using MLflow, Airflow, and monitoring tools like Prometheus/Grafana/Nagios.

THEORY :

Continuous Monitoring

Continuous monitoring is the practice of detecting, reporting and responding to problems in an infrastructure while it is in use. The work of a DevOps team does not end when an application is deployed to a server, because a deployment that was healthy at release can fail later from a full disk, an exhausted process table, a stopped service or a network fault. Continuous monitoring closes that gap by checking the infrastructure repeatedly on a schedule, recording the result of every check, and notifying the people responsible as soon as a value crosses a defined limit. The purpose is to find a fault while it is still degrading the system rather than after it has caused an outage.

Nagios

Nagios is an open-source monitoring system used to watch hosts, services and network devices. It allows an organisation to identify and resolve infrastructure problems before they affect critical business processes. Nagios detects network and server issues, helps locate the root cause of a

problem so that a permanent fix can be applied, monitors and troubleshoots server performance, and indicates when systems are due for an upgrade before an outdated component causes a failure. In doing so it maintains the security and availability of the services it monitors.

Nagios Core Architecture

Nagios Core consists of a scheduling daemon, a collection of plugins and a web interface. The daemon reads a set of object definition files that describe what is to be monitored, maintains a queue of checks due to be executed, and runs each check at its appointed time. The result of every check is written to a status file, and a set of CGI programs render that file as the web interface through which the current state of the infrastructure is viewed. Configuration is therefore entirely text based, and any change to the object files requires the daemon to be validated and reloaded before it takes effect.

Plugins and Check Results

Nagios does not itself know how to measure a disk, a processor or a running process. Every measurement is performed by a plugin, which is a small external program that Nagios executes. The plugin performs the measurement and communicates its verdict through its exit status: zero for OK, one for WARNING, two for CRITICAL and three for UNKNOWN. Along with the exit status the plugin prints a single line of text describing the result, and optionally performance data after a vertical bar for graphing. The limits at which a plugin should report a warning or a critical condition are supplied to it as arguments, conventionally -w and -c. Because the interface between Nagios and a plugin is only an exit code and a line of text, anything that can be expressed as a script can be monitored.

Hosts, Services and Templates

The objects to be monitored are described as hosts and services. A host represents a machine and carries the address at which it is reached; a service represents one thing being checked on that host and names the check command to be used. Repeated settings are factored out into templates, so that a host definition need only inherit a template such as linux-server or windows-server and supply what is specific to it. Each service also defines how often it is checked, how often it is retried after a failure, and how many attempts must fail before the failure is believed. Until that number of

attempts is reached the state is called soft; once it is reached the state becomes hard and notifications are sent. This distinction prevents a single momentary failure from raising a false alarm.

NRPE and the Remote Agent

A plugin executed on the Nagios server can test a remote host across the network, but it cannot read a value that exists only inside that host, such as its disk usage or its process count. NRPE, the Nagios Remote Plugin Executor, solves this. An NRPE agent is installed on the machine to be monitored and listens on TCP port 5666. When the Nagios server needs a value from that machine it sends the name of a command; the agent looks the name up in its own configuration, executes the corresponding plugin locally, and returns the exit status and output to the server. The agent will only answer requests from addresses listed as allowed hosts in its configuration, so the Nagios server address must be permitted before any check will succeed. On Windows the equivalent agent is NSClient++, which serves the same purpose and also speaks the NRPE protocol.

Active and Passive Checks

A check initiated by the Nagios daemon on its own schedule is called an active check, and every check in this experiment is of that kind. Nagios can also accept a passive check, in which an external process submits a result to the daemon through its command file. The same command file is used to instruct a running daemon to perform an action immediately, such as forcing a scheduled check to run at once instead of waiting for its next interval.

Nagios in a Container

Nagios Core is written for Linux and is not installed natively on Windows or macOS. The standard way to run it on either platform is inside a Docker container, which supplies the Linux userland, the Apache server and the Nagios binaries as a single ready-made image. The container is started with its web port published to a port on the host, so the web interface is reached at localhost even though Nagios is running in an isolated Linux environment. The machine that hosts the container can then be treated as an ordinary remote host and monitored through an agent.

Prometheus and Grafana

Prometheus and Grafana are the alternative tools named in the aim, and they follow a different model. Prometheus periodically scrapes numeric metrics that an application exposes at an HTTP endpoint and stores them as a time series, which can then be queried and used to define alerting

rules. Grafana reads such a data source and presents it as dashboards of graphs. The difference in approach is that Nagios records the state of a check at a point in time and is oriented towards the question of whether something is currently broken, whereas Prometheus retains the history of a measurement and is oriented towards how a value has behaved over time. Nagios was chosen for this experiment because it demonstrates host and service monitoring together with a remote agent.

Monitoring Flow:

Nagios daemon reaches the scheduled check time → the check command is executed, either locally or through the remote agent → the plugin measures the value and exits with a status code → Nagios compares the result with the previous state and records it → a repeated failure becomes a hard state and a notification is raised → the current state of every host and service is displayed in the web interface.

EXECUTION :

The experiment was performed on macOS running on Apple Silicon, whereas the procedure given in the laboratory handout is written for Windows PowerShell. Two steps therefore required a substitution rather than only a change of syntax, and both are stated at the point where they occur. The Nagios Core server was run as a Docker container, and the machine hosting that container was itself registered in Nagios as the remote host to be monitored.

Step 1: Verify the Docker Installation

Nagios Core cannot be installed natively on the host operating system, so Docker Desktop was used to provide a Linux environment for it. The installation was verified from the terminal before continuing. The WSL2 commands of the handout were not required, because WSL2 exists only on Windows and Docker Desktop supplies its own Linux virtual machine on macOS.

```
$ docker --version
Docker version 29.4.3, build 055a478
$ docker info --format '{{.ServerVersion}} / {{.OperatingSystem}}'
29.4.3 / Docker Desktop
```

[Screenshot: terminal showing the Docker version]

Step 2: Pull and Run the Nagios Core Image

A ready-made Nagios Core image was pulled and started as a container named nagios4, with port 80 of the container published to port 8080 of the host so that the web interface could be reached from a browser. The image is published for the amd64 architecture only, while the machine used is arm64, so the platform was stated explicitly and the container runs under emulation. Port 8080 was confirmed to be free before the container was started.

```
$ docker pull --platform linux/amd64 jasonrivers/nagios:latest
$ docker run --name nagios4 --platform linux/amd64 -d -p 8080:80
jasonrivers/nagios:latest
$ docker ps --filter name=nagios4
NAMES     STATUS          PORTS
nagios4   Up 15 seconds   0.0.0.0:8080->80/tcp
```

[Screenshot: docker ps output showing the nagios4 container running]

Step 3: Set the Nagios Administrator Password

The web interface is protected by Apache basic authentication, and the password for the nagiosadmin user is held in a htpasswd file inside the container. The password was set with the htpasswd utility, executed inside the running container.

```
$ docker exec nagios4 htpasswd -c -b /opt/nagios/etc/htpasswd.users
nagiosadmin 123456
Adding password for user nagiosadmin
```

Step 4: Access the Nagios Web Interface

The interface was opened in a browser on the host machine at the published port. The browser requests the user name and password set in the previous step. Authentication was confirmed from the terminal first: a request without credentials is refused with status 401, and the same request with the credentials returns 200.

```
http://localhost:8080/nagios/
username: nagiosadmin
password: 123456

$ curl -s -o /dev/null -w '%{http_code}' http://localhost:8080/nagios/
401
$ curl -s -o /dev/null -w '%{http_code}' -u nagiosadmin:123456
http://localhost:8080/nagios/
200
```

[Screenshot: Nagios web interface after logging in as nagiosadmin]

Step 5: Install the Remote Agent on the Monitored Host

The handout installs NSClient++ on the host to be monitored. NSClient++ is distributed only as a Windows installer and has no build for macOS, so the agent it stands for was installed directly instead: NRPE itself, together with the standard plugin collection that the agent executes. This is a substitution of the same component rather than of the method, because NSClient++ is used in the handout only to answer NRPE requests.

```
$ brew install nrpe monitoring-plugins
nrpe: stable 4.1.3 (bottled)
```

The agent binary is installed at /opt/homebrew/opt/nrpe/bin/nrpe and the check plugins at /opt/homebrew/sbin/.

Step 6: Configure the Remote Agent

The agent refuses requests from any address that is not listed in its configuration, so the address of the Nagios server was permitted first. A container on this platform reaches the host through the address of the Docker virtual machine rather than through an address on the local network, so the source range of that virtual machine was allowed instead of the single fixed address used in the handout. The original configuration file was kept as nrpe.cfg.bak before editing.

```
$ nano /opt/homebrew/etc/nrpe.cfg
allowed_hosts=127.0.0.1,::1,192.168.65.0/24,172.17.0.0/16
```

Three of the command definitions supplied with the agent are written for Linux and had to be corrected before they would return a meaningful result on this system. The disk check referred to the device /dev/hda1, which does not exist here, and was redefined to check the root filesystem. The load thresholds of .15, .10 and .05 are far below normal load and reported a critical state permanently. The process count thresholds of 150 and 200 are exceeded by this operating system even when idle, and were raised to values appropriate to it.

```
command[check_users]=/opt/homebrew/sbin/check_users -w 5 -c 10
command[check_load]=/opt/homebrew/sbin/check_load -r -w 5,4,3 -c 10,8,6
command[check_root]=/opt/homebrew/sbin/check_disk -w 20% -c 10% -p /
command[check_total_procs]=/opt/homebrew/sbin/check_procs -w 600 -c 800
```

The agent was then started as a background service and confirmed to be listening on its port.

```
$ brew services start nrpe
```

```
$ lsof -nP -iTCP:5666 -sTCP:LISTEN
nrpe   52574 dhruv   5u  IPv4   TCP *:5666 (LISTEN)
```

Step 7: Verify Connectivity with check_nrpe

Before defining any object in Nagios, the connection from the server to the agent was tested with the check_nrpe plugin from inside the container. Called without a command it returns the version of the agent, which proves that the request was accepted and answered. The address host.docker.internal is how a container resolves the machine hosting it, and it replaces the fixed remote address used in the handout.

```
$ docker exec nagios4 /opt/nagios/libexec/check_nrpe -H host.docker.internal
NRPE v4.1.3
```

Each individual command was then requested over the same connection, confirming that the agent executes the plugins locally and returns their output.

```
$ docker exec nagios4 /opt/nagios/libexec/check_nrpe -H host.docker.internal -
c check_users
USERS OK - 2 users currently logged in |users=2;5;10;0
$ ... -c check_load
OK - load average per CPU: 0.34, 0.29, 0.23
$ ... -c check_root
DISK WARNING - free space: / 38056 MiB (16.28% inode=100%)
$ ... -c check_total_procs
PROCS OK: 445 processes
```

[Screenshot: check_nrpe returning the agent version and the individual check results]

Step 8: Define the Host and its Services in Nagios

The host and the services to be monitored were described in an object file inside the container. Two adjustments were needed that the handout does not mention. The directory in which such files are placed did not exist in this image and the line that includes it was commented out in the main configuration, so the line was enabled and the directory created. The image also ships no definition of the check_nrpe command, so that definition was added alongside the host.

```
cfg_dir=/opt/nagios/etc/servers            # enabled in nagios.cfg
$ docker exec nagios4 mkdir -p /opt/nagios/etc/servers
```

The host inherits the linux-server template and is addressed at the machine hosting the container. The PING service is defined as in the handout, and four services were added on top of it that obtain their values from the remote agent.

```
define command{
    command_name    check_nrpe
    command_line    $USER1$/check_nrpe -H $HOSTADDRESS$ -c $ARG1$
}

define host {
    use              linux-server
    host_name        Mac_Host_001
    alias            macOS Host 001 (Docker host)
    address          host.docker.internal
    register         1
}

define service{
    host_name               Mac_Host_001
    service_description     PING
    check_command           check_ping!100.0,20%!500.0,60%
    max_check_attempts      2
    check_interval          2
    retry_interval          2
    check_period            24x7
    contact_groups          admins
    notification_interval   2
    notification_period     24x7
    notifications_enabled   1
    register                1
}

define service{
    use                     generic-service
    host_name               Mac_Host_001
    service_description     Root Partition
    check_command           check_nrpe!check_root
    contact_groups          admins
}
```

The remaining three services, Current Users, CPU Load and Total Processes, were defined in the same form with their own check_nrpe command. The configuration was then validated and the container restarted so that the daemon would read it.

```
$ docker exec nagios4 /opt/nagios/bin/nagios -v /opt/nagios/etc/nagios.cfg
Total Warnings: 0
Total Errors:   0
Things look okay - No serious problems were detected during the pre-flight
check
$ docker restart nagios4
```

[Screenshot: output of the configuration verification showing zero warnings and zero errors]

Step 9: View the Monitored Host in the Web Interface

The web interface was opened again and the new host was found under Current Status, both as a host and as a list of its services. A service is shown as pending until its first check has run, so the first results were obtained at once by submitting a forced check to the command file of the daemon rather than waiting for the next scheduled interval.

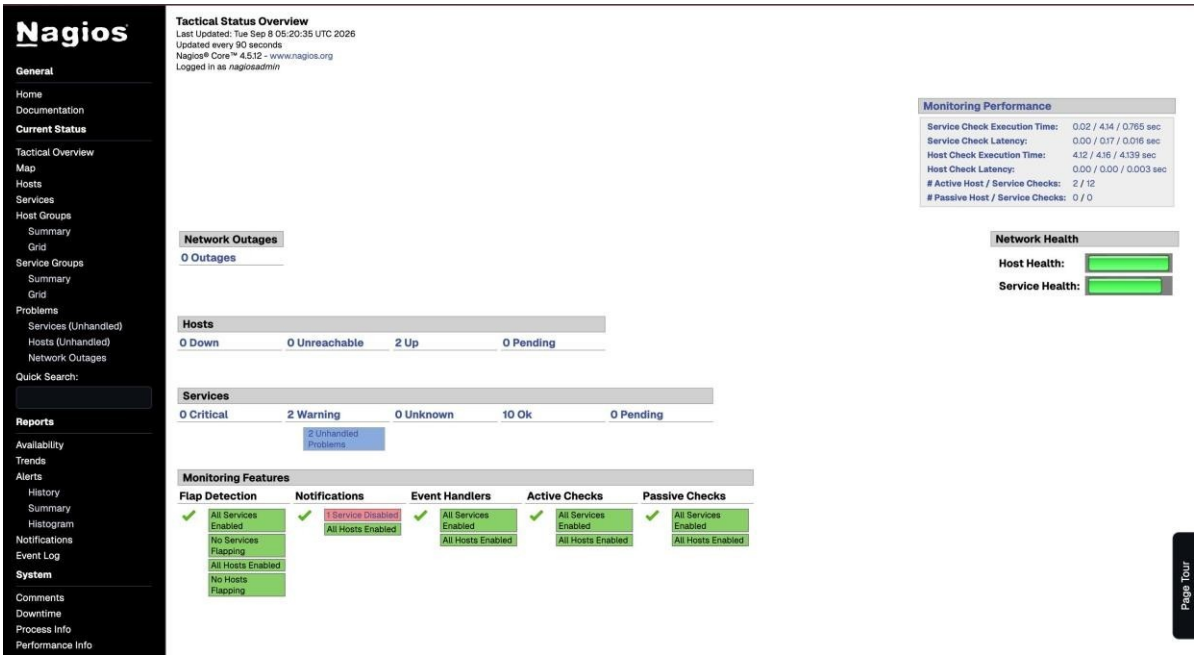
```
$ docker exec nagios4 sh -c "printf '[%s]
SCHEDULE_FORCED_SVC_CHECK;Mac_Host_001;PING;%s\n' \
      $(date +%s) $(date +%s) > /opt/nagios/var/rw/nagios.cmd"
```

The host was reported as UP, and the five services returned the states listed below.

Table 1: State of host Mac_Host_001 in the Nagios web interface

Service	Check command	State	Plugin output
PING	`check_ping`	OK	`PING OK - Packet loss = 0%, RTA = 0.44 ms`
Current Users	`check_nrpe!check_users`	OK	`USERS OK - 2 users currently logged in`
CPU Load	`check_nrpe!check_load`	OK	`OK - load average per CPU: 0.19, 0.25, 0.25`
Total Processes	`check_nrpe!check_total_procs`	OK	`PROCS OK: 541 processes`
Root Partition	`check_nrpe!check_root`	WARNING	`DISK WARNING - free space: / 36963 MiB (15.81% inode=100%)`

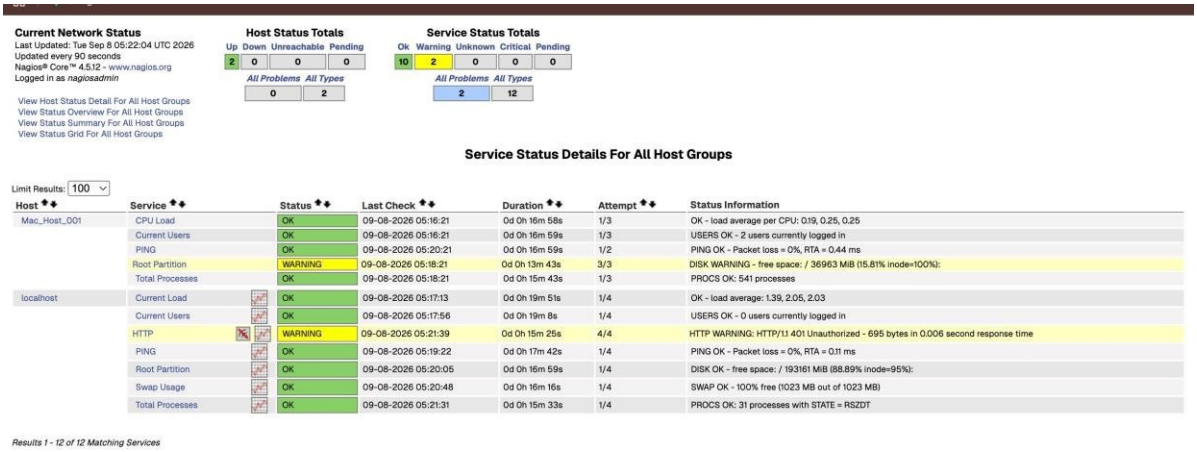
The Tactical Overview summarises the whole monitored infrastructure on one page. It reports two hosts up and none down, unreachable or pending, and twelve services of which ten are OK, two are in a warning state and none are critical or pending. The page also confirms the version of Nagios Core in use, 4.5.12, the account logged in, and that the display refreshes every ninety seconds.



The Monitoring Performance panel of that page reports the count of active host and service checks as 2 and 12, and the count of passive host and service checks as 0 and 0, which confirms that every check in this experiment is an active check initiated by the daemon itself. The panel also gives the minimum, maximum and average execution time of the checks. Service checks averaged 0.765 seconds while host checks averaged 4.139 seconds, and the difference is explained by the host check command, which sends five ping packets and therefore has to wait for all five to return. Check latency, the delay between the time a check was due and the time it actually ran, averaged 0.016 seconds for services, showing that the daemon was keeping up with its schedule.

Under Monitoring Features the panel reports one service with notifications disabled. That service is the HTTP check of localhost, for which the configuration supplied with the image suppresses notifications; its warning is therefore recorded and displayed but raises no alert.

The detailed listing of every service of both hosts is shown below.



The listing shows two hosts rather than one. Mac_Host_001 is the host defined in the previous step, and localhost is the container itself, which the image monitors out of the box through the configuration file supplied with it. Between them they account for the twelve services reported at the top of the page, of which ten are OK and two are in a warning state.

The warning reported for the root partition of Mac_Host_001 is a genuine reading and not a fault in the configuration: the disk of the monitored machine held only 15.81 per cent free space, which is below the twenty per cent warning threshold given to the plugin. The attempt column reads 3/3 for that service, meaning the check had failed the maximum number of times allowed and the state had therefore become hard, whereas the services that are OK read 1/3. An earlier run of the same check returned 16.27 per cent free and a process count of 445 rather than 541, which shows that the values are measured afresh at every interval rather than reported from a cached result.

The second warning belongs to localhost and is a consequence of step 3. The HTTP check supplied with the image requests the web server inside the container and expects it to answer normally, but the interface was placed behind a password in that step, so the server now answers 401 Unauthorized and the plugin reports a warning. It is an accurate report of what the check was asked to measure rather than a fault, and it illustrates that a check keeps testing the condition it was originally given even after the system around it has been changed deliberately.

CONCLUSION :

The experiment successfully demonstrated continuous monitoring of an infrastructure using Nagios. A Nagios Core server was run as a Docker container with its web interface published to the host, the administrator account was configured, and the interface was used to observe the state of the monitored infrastructure. A remote agent was installed on the machine to be monitored, restricted to accept requests only from the Nagios server, and verified with the check_nrpe plugin before any object was defined. The host and five services were then described as object definitions, validated with the pre-flight check, and confirmed to be monitored in the web interface.

The experiment made clear that Nagios performs no measurement of its own. Every value comes from a plugin that Nagios executes and that reports its verdict through an exit status, and every value that exists only inside a remote machine has to be fetched through an agent on that machine. This separation is what allows the same monitoring server to watch a disk, a process count and a network path without containing any code specific to them, and it is also why a threshold supplied to a plugin has to suit the system being measured. Several of the default command definitions were written for a Linux server and reported a permanently critical state on the system used here until their thresholds were corrected.

Adapting the procedure to a different operating system was instructive in itself. Most of the steps differed only in the syntax of the command, but two required an actual substitution: the Nagios server had to be containerised because Nagios Core does not run natively outside Linux, and the Windows agent NSClient++ had to be replaced with NRPE, the agent whose protocol NSClient++ implements. Recognising which component a step depends on, rather than which product it names, was what allowed the procedure to be completed without departing from its intent.

Overall, the experiment provided practical knowledge of continuous monitoring, of the plugin and exit-status model on which Nagios is built, of remote monitoring through NRPE and its allowed-hosts restriction, and of the object definition, validation and reload cycle by which a Nagios configuration is changed. The distinction between the state-based model of Nagios and the time-series model of Prometheus and Grafana was also examined, clarifying when each kind of monitoring tool is appropriate.